

기획 변경 사항을 반영하여 **[토템 슬라이딩 퍼즐 시스템 기획서 v1.1]**로 업데이트했습니다.

변경 핵심 사항:

- 기존: 장애물 토템은 피격 시 1칸만 이동.
- 변경: 장애물 토템도 오브젝트 토템과 동일하게 직선 방향으로 끝까지 미끄러짐(Sliding).

이로 인해 장애물 토템 또한 '멈추는 위치'를 전략적으로 고려해야 하는 요소로 변경되었습니다.

시스템 기획서: 고대 토템 슬라이딩 퍼즐 (v1.1)

1. 개요 (Overview)

- 시스템 명: 토템 슬라이딩 퍼즐 시스템 (Totem Sliding Puzzle System)
- 목표: 플레이어는 '차징 공격'을 이용해 토템들을 이동시키고, 최종적으로 [오브젝트 토템]을 [목표 타일] 위에 위치시켜야 한다.
- 핵심 변경점: 모든 토템(오브젝트/장애물)이 물리적 충격에 의해 끝까지 미끄러지는 슬라이딩 방식을 채택.

2. 월드 구성 요소 (World Components)

2.1. 퍼즐 그리드 (Puzzle Grid)

- 규격: 5×5 Grid System (총 25개 타일)
- 좌표계: $(0,0) \sim (4,4)$ 로컬 좌표 사용.

2.2. 등장 오브젝트 (Puzzle Objects)

오브젝트 명	역할 및 특징	이동 규칙 (변경됨)	파괴 및 내구도
오브젝트 토템 (Object Totem)	[Key Object] 목표 지점으로 옮겨야 함.	피격 시: 직선 방향으로 끝까지 미끄러짐 (Sliding). 벽이나 다른 토템	파괴 가능 $(HP \leq 0)$ 시 파괴) 파괴 시 퍼즐 리셋.

		충돌 시 정지.	
장애물 토템 (Obstacle Totem)	[Movable Wall] 이동 가능한 벽 역할. 자신이 벽이 되거나 길을 틈.	피격 시: 직선 방향으로 끝까지 미끄러짐 (Sliding) . 벽이나 다른 토템 충돌 시 정지.	파괴 불가 (Immortal) 내구도 감소 로직 없음.

3. 핵심 메커니즘 (Core Mechanics)

3.1. 통합 이동 로직 (Unified Movement Logic)

이제 두 토템은 동일한 이동 알고리즘을 공유합니다. 단, 내구도 처리만 다릅니다.

[공통 슬라이딩 프로세스]

1. **Trigger:** 플레이어의 차징 공격(**Charged Attack**) 히트 박스와 충돌.
2. **Direction:** 공격 벡터를 기반으로 4방향(상/하/좌/우) 중 하나 결정.
3. **Action:** 결정된 방향으로 가속 이동 시작 (State: Sliding).
4. **Stop Condition (정지 조건):**
 - 그리드 경계(Wall) 도달.
 - 다른 정지 상태(Idle)의 토템과 충돌.
5. **Snap:** 정지 시 해당 그리드 중앙 좌표로 보정.

3.2. 내구도 로직 (Durability Logic) - 오브젝트 토템 전용

- 장애물 토템: 피격 및 이동 횟수에 제한이 없으며 파괴되지 않음.
- 오브젝트 토템:
 - 초기값 Max_HP 설정.
 - 이동(Sliding)이 시작될 때마다:
 $Current_HP = Current_HP - 1$
 - $Current_HP \leq 0$
조건 만족 시: 오브젝트 파괴(**Destroy**) 및 퍼즐 실패(**Fail**) 처리.

3.3. 승리 조건 (Winning Condition)

- 모든 Object Totem이 지정된 Target Tile 위에 위치하고, 움직이지 않는(Idle) 상태일 것.
- 성공 시: TargetInteractionObject의 Bool 값을 True로 변경.

4. 상태 정의 (State Definition)

두 토템 모두 이동 방식이 같아졌으므로 상태 머신이 유사해집니다.

4.1. 토템 공통 FSM (Totem Common FSM)

상태 (State)	설명 (Description)	오브젝트 토템 동작	장애물 토템 동작
Idle	대기 상태 (충돌체 역할 O)	피격 대기	피격 대기
Hit	피격 경직 (이동 방향 계산)	HP - 1 차감	HP 차감 없음
Sliding	이동 중 (충돌체 역할 X)	이동 (무적 판정)	이동 (무적 판정)
Destroyed	파괴됨 (비활성화)	파괴 이펙트 & 리셋 트리거	(진입 불가 상태)

5. 예외 처리 및 엣지 케이스 (Exceptions & Edge Cases)

장애물 토템이 슬라이딩하게 되면서 충돌 관련 예외 처리가 더 중요해졌습니다.

5.1. 이동 중인 토템 간의 충돌 (Collision while Sliding)

- 상황: 토템 A가 이동 중인데, 이동 경로상에 이미 움직이고 있는 토템 B가 끼어드는 경우. (타이밍 이슈)
- 처리:
 1. 선착순 우선: 먼저 해당 타일의 점유권(Occupancy)을 가진 토템이 우선권을 가짐.
 2. 동시 진입 시: 둘 다 멈추고 제자리에서 살짝 튕겨나는(Bump) 연출 후 Idle로 전환.
 - 기획 의도: 퍼즐의 복잡도를 낮추기 위해, 원칙적으로 **한 번에 하나의 토템만 움직이는 것**을 권장하나, 시스템상으로는 충돌 시 즉시 정지를 원칙으로 함.

5.2. "제자리" 피격 (Zero Distance Hit)

- 상황: 토템이 벽에 딱 붙어 있는데, 벽 쪽으로 미는 공격을 한 경우.
- 처리:
 - 이동 불가능하므로 Sliding 상태로 진입하지 않음.
 - 오브젝트 토템: 이동하지 않았으므로 HP를 차감하지 않음. (단순 실수 방지)
 - 가벼운 '흔들림' 애니메이션만 재생.

5.3. 샌드위치 상태 (Sandwiched Player)

- 상황: 플레이어가 벽과 토템 사이에 있는데 토템을 당기거나 밀어서 플레이어가 끼이는 경우.
- 처리: 플레이어 캐릭터의 충돌체(Collider)는 토템의 이동을 막지 못함. 플레이어는 강제로 옆 타일로 밀려나거나(Pushed), 데미지를 입고 낙백됨.

5.4. 리셋(Reset) 시점의 처리

- 상황: 오브젝트 토템이 파괴되어 퍼즐이 리셋될 때, 장애물 토템의 위치.
- 처리: 퍼즐의 초기 배치가 해법의 기준이 되므로, 장애물 토템도 반드시 초기 위치(Initial Position)로 복귀해야 함.

6. 데이터 구조 (Data Structure)

변경 사항을 반영한 데이터 클래스 설계입니다.

C#

```
public enum TotemType { Object, Obstacle }
```

```
[System.Serializable]
```

```
public class TotemData {  
    public TotemType type;      // 토템 타입 구분  
    public Vector2Int startPos; // 초기 그리드 좌표 (x, y)
```

```
    [Header("Object Totem Only")] // 오브젝트 토템일 때만 사용  
    public int maxDurability;      // 내구도  
    public Vector2Int targetPos;   // 목표 좌표  
}
```

```
[System.Serializable]
```

```
public class PuzzleLevelData {  
    public int levelID;  
    public List<TotemData> totems; // 맵에 배치된 모든 토템 리스트  
    public GameObject doorObject;  // 성공 시 열릴 문  
}
```

기획자의 코멘트 (Update)

- 장애물 토텐도 슬라이딩하게 변경됨에 따라, "장애물 토텐을 먼저 특정 위치로 슬라이딩 시켜 벽(Wall)을 만든 뒤, 오브젝트 토텐을 그 벽에 부딪히게 하여 멈추는" 식의 연계 플레이가 핵심 퍼즐 기믹이 될 것입니다.
- 레벨 디자인 시, 장애물 토텐을 잘못된 위치로 슬라이딩 시키면(예: 구석에 박힘) 퍼즐을 클리어할 수 없는 **Deadlock(교착 상태)**이 발생할 수 있습니다. 이를 위해 플레이어가 언제든지 퍼즐을 **수동 초기화** 할 수 있는 기능/버튼이 필수적으로 요구됩니다.

제가 해드릴 수 있는 다음 단계:

1. 이 기획에 맞춰 토텐의 **이동을 담당하는 스크립트(C#)**의 의사 코드(Pseudo-code)를 작성해 드릴까요?
2. 아니면, 발생할 수 있는 **구체적인 퍼즐 레벨 디자인 예시(5x5 그리드 배치도)**를 그려볼까요?